

Ω-LRD3：自索引列图

定义与证明资料 / 中文版

发布定义，2026-09-13。PDF · NER 展开器 · Python · 良序证明。

数字 3 是版本号。行标是同一系统的有限表达式。关键约定是：种子每级只追加一列，生成包指标取到 b ，不是 $b + 1$ 。本文自包含，不依赖其他 Ω-LRD 版本。

1. 有限嵌套语法

表达式是有限、无环、递归构造的列图。有限图 $G = (m, E)$ 的列号为 $0, \dots, m - 1$ ，关系为

$$(K, q, p, j), \quad 0 \leq q \leq p < j < m,$$

其中行标 K 是已经构造的同类有限图。四个字段依次是行标、根列、父列、子列。允许空图和空列。每个行标都是严格子表达式；禁止循环引用，禁止把外顶端用作行标。共享相同不可变子表达式只是实现优化。

要求根闭包：

$$(K, q, p, j) \in E \implies (K, u, p, j) \in E \quad (0 \leq u \leq q).$$

闭包 $\downarrow$ 增加这些较小根，不会补全所有较低行标。重复关系忽略，空列保留。零列图为 0，显示 $\emptyset$ ；由 a 个空列构成的图作为有限数行标 a 。所以 $[]$ 是一， $[] []$ 是二，均非零。

每个方括号是一列。其中（行标，父列，最大根）代表从零到最大根的全部关系；子列由位置确定。同一（行标，父列）分组合并取最大根。有限数行标直接写数字，其他行标用花括号包住完整列表。例如：

```
[][[(1,0,0)][{[][[(1,0,0)]},1,1)]
```

最后一个关系的行标是两列图 $[][[(1,0,0)]]$ ，父列为 1，最大根为 1。花括号是嵌套语法，不是外加的序数运算。

2. 递归列序

每列的最大根三元组 (K, p, q) 按 (p, K, q) 降序排列。按该键逐项比较关系，再比较列内列表，最后从最左列开始比较整图。各级列表都以真前缀较小。行标递归使用相同规则比较。

每次递归进入严格子表达式，因此比较是有限结构递归。最大根与全部根表示得到相同结果。项的身份是规范化图的相等，不是展开历史或计数序列相等。

外加顶端 Limit，大于所有有限图。它不是有限图，不能出现在行标内。

3. 有限图的基本列

指标 n 为非负整数。规定 $0[n] = 0$ ，不将此恒等式视为严格下降步。对非空图令 $x = m - 1$ ，以 ∂G 表示删末列及以它为子列的所有关系。规定 $G[0] = \partial G$ ；若末列无关系，则全部 $G[n] = \partial G$ 。

余下考虑 $n > 0$ 。按 (K, q, p) 取末列最大关系，写成 (K, r, c, x) 。控制优先级为**行标、根、父列**；比较优先级为**父列、行标、根**。令

$$\ell = x - c > 0, \quad \phi_b(i) = \begin{cases} i & i < c, \\ i + b\ell & i \geq c, \end{cases} \quad E^- = \{(H, q, p, j) \in E : j < x\}.$$

对关系施加 ϕ_b 时移动根、父列、子列，行标表达式不变。有限生成包为

$$P_b(K) = \begin{cases} \emptyset & K \text{ 的全部列都为空列,} \\ \{0\} \cup \{K[t] : 0 \leq t \leq b\} & \text{否则.} \end{cases}$$

这里 $K[t]$ 就按本节规则展开。递归依据是严格子表达式 K ，不是未来可能巨大的展开次数，因此单次展开是有限结构算法。有限数行标（包括零）不生成行标包。

对 $0 \leq b < n$ 定义

$$\begin{aligned} U_b &= \{(H, u, \phi_b(p), x + b\ell) : (H, q, p, x) \in E, \quad 0 \leq u \leq \phi_b(q), \\ &\quad H < K \text{ 或 } (H = K \text{ 且 } u < \phi_b(r))\}, \\ V_b &= \{(D, u, \phi_b(c), x + b\ell) : D \in P_b(K), \quad 0 \leq u \leq \phi_b(c)\}. \end{aligned}$$

最终

$$G[n] = \left(x + n\ell, \downarrow \left(\bigcup_{b=0}^n \phi_b(E^-) \cup \bigcup_{b=0}^{n-1} (U_b \cup V_b) \right) \right).$$

源块 $[c, x]$ 保留原位，并向右追加 n 份。规范化去重并补齐根。增加 n 只追加完整列； $G[n]$ 是 $G[n+1]$ 的前缀，原末列有关系时为真前缀。旧末列之前的列不变。每个非零有限展开在递归列序下严格变小。

4. 种子与顶端

定义 $A_0 = 0$ 、 $A_1 = []$ 。对 $n \geq 1$ ，恰好追加一列：

$$A_{n+1} = A_n \parallel [(A_n, n-1, n-1)], \quad \text{Limit}[n] = A_n.$$

新三元组采用列表字段顺序（行标,父列,最大根）。行标是整个旧式 A_n ，不是新式 A_{n+1} 。最大根 $n-1$ 包含零至 $n-1$ 的全部根。

因此 A_n 恰有 n 列。 A_0 、 A_1 的嵌套深度为零；此后最大行标嵌套深度为 $n-1$ 。对每个 $n \geq 0$,

$$A_{n+1}[0] = A_n.$$

所以顶端基本列本身也是列前缀链。种子除首列外，不在人为关系列之间插空列。普通展开仍可能产生空列，它们必须保留。

5. 标准域与定理范围

本记号包含 Limit 及从它经过有限次展开可达的全部有限图。等价地，有限标准域是各个种子 A_n 的展开后代域之并。能解析或者满足索引约束，不足以说明是标准式。

链接的纸面证明在 $KP_\omega + \exists$ 不可数序数内证明标准递归列序（包括 Limit ）良序。证明按结构深度归纳；对每个有限起点，把有限多个直接行标的后代锥取并，得到固定行标池，再套用固定行标的有限需求反射定理。配套 Lean 最终定理针对这个标准域；它是通常经典 Lean 证明，不是弱集合论推导在 Lean 内的编码。

该结论不把列良序推广到所有原始有限图。本文也不声称与 Y、RPD、LRD 的序型比较或最小可能的公理强度。

6. 显示、计数与 NER 输入

NER 默认是无损单行列表，额外等价显示是计数序列。本版不绘制山脉图，不用省略号隐藏嵌套行标。原样保留的脚本使用中文显示标签。

对一列固定左前缀，将它作为末列，取任意正指标展开，再丢掉新增列。重复直到保留位置无关系，最后删掉该列。计数包括最后删除；空关系列计数为一。只有第零接缝影响保留位置，所以非有限 K 的局部行标包是 $\{0, K[0]\}$ ，其

中 $K[0]$ 就是删去 K 的末列。

计数不是比较键，也不唯一编码图。JS 用精确 BigInt 累加，没有有符号 64 位计数上限。若保护计算未完成，显示会明确说明，不会将下界冒充精确答案。

输入	含义或计数序列
Limit	外顶端
A0	0
A1	1
A2	1, 2
A3	1, 2, 10
A4	1, 2, 10, 47
A5	1, 2, 10, 47, 240
A6	1, 2, 10, 47, 240, 1357

在 NER 自定义记号入口导入 `Omega-LRD3.ne-rewritten.js` 并选择 Ω -LRD3。可输入 `A3[2][1]`、`Limit[3]`、自然数或完整列表。Tn 是 An 的别名。花括号包住有限行标表达式，例如 `[][(A2),0,0]`；禁止 `{Limit}`。FS、FS_alter、FS_short 完全相同。

当前脚本在同一事件循环任务内共享约一秒的检查点预算。列宽、嵌套、文本和缓存限制是实现保护，不是数学界限，也不是硬实时保证。资源超限不构成新的零或后继情形。

7. 独立 Python 展开器

只需 Python 3.10+ 和标准库。在本目录运行：

```
python omega_lrd3.py 3 2 1
```

它计算 `Limit[3][2][1]`。作为库使用：

```
from omega_lrd3 import OmegaLRD3, ZERO, TOP
a = OmegaLRD3.seed(3)
assert a == TOP[3]
assert a[0] == OmegaLRD3.seed(2)
assert OmegaLRD3.integer(2).is_integer
b = a[2]
assert b < a
```

使用 `OmegaLRD3(columns)` 构造自定义图，每列包含（行标图,父列,最大根）元组。行标必须是有限的 `OmegaLRD3` 对象，不能用 TOP。构造器规范化最大根并检查索引，但不判定标准性。整数完全精确，没有内置时间或宽度限制；深层嵌套可能触发 Python 递归上限，极大输出可能耗尽内存。发布包的有界测试对照较早的独立嵌套元组参考和原样 JS，并非覆盖全部可能输入的证明。